
MLP Coursework 1

S2212195

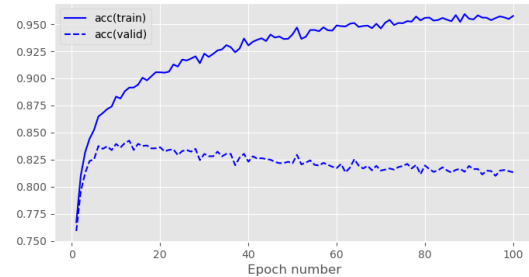
Abstract

In this report we study the problem of overfitting, which is the training regime where performance increases on the training set but decrease on validation data. Overfitting prevents our trained model from generalizing well to unseen data. We first analyse the given example and discuss the probable causes of the underlying problem. Then we investigate how the depth and width of a neural network can affect overfitting in a feedforward architecture and observe that increasing width and depth tend to enable further overfitting. Next we discuss how two standard methods, Dropout and Weight Penalty, can mitigate overfitting, then describe their implementation and use them in our experiments to reduce the overfitting on the EMNIST dataset. Based on our results, we ultimately find that both dropout and weight penalty are able to mitigate overfitting. Finally, we conclude the report with our observations and related work. Our main findings indicate that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

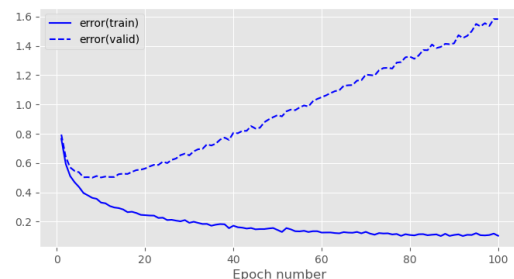
1. Introduction

In this report we focus on a common and important problem while training machine learning models known as overfitting, or overtraining, which is the training regime where performances increase on the training set but decrease on unseen data. We first start with analysing the given problem in Figure 1, study it in different architectures and then investigate different strategies to mitigate the problem. In particular, Section 2 identifies and discusses the given problem, and investigates the effect of network width and depth in terms of generalization gap (see Ch. 5 in Goodfellow et al. 2016) and generalization performance. Section 3 introduces two regularization techniques to alleviate overfitting: Dropout (Srivastava et al., 2014) and L1/L2 Weight Penalties (see Section 7.1 in Goodfellow et al. 2016). We first explain them in detail and discuss why they are used for alleviating overfitting. In Section 4, we incorporate each of them and their various combinations to a three hidden layer¹ neural network, train it on the EMNIST dataset, which contains 131,600 images of characters and digits,

¹We denote all layers as hidden except the final (output) one. This means that depth of a network is equal to the number of its hidden layers + 1.



(a) accuracy by epoch



(b) error by epoch

Figure 1. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for the baseline model.

each of size 28x28, from 47 classes. We evaluate them in terms of generalization gap and performance, and discuss the results and effectiveness of the tested regularization strategies. Our results show that both dropout and weight penalty are able to mitigate overfitting.

Finally, we conclude our study in Section 5, noting that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

2. Problem identification

Overfitting to training data is a very common and important issue that needs to be dealt with when training neural networks or other machine learning models in general (see Ch. 5 in Goodfellow et al. 2016). A model is said to be overfitting when as the training progresses, its performance on the training data keeps improving, while its is degrading on validation data. Effectively, the model stops learning related patterns for the task and instead starts to memorize specificities of each training sample that are irrelevant to new samples. Overfitting leads to bad generalization per-

# Hidden Units	Val. Acc.	Train Error	Val. Error
32	79.2	0.538	0.681
64	81.0	0.332	0.686
128	81.0	0.154	0.945

Table 1. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network widths on the EMNIST dataset.

formance in unseen data, as performance on validation data is indicative of performance on test data and (to an extent) during deployment.

Although it eventually happens to all gradient-based training, it is most often caused by models that are too large with respect to the amount and diversity of training data. The more free parameters the model has, the easier it will be to memorize complex data patterns that only apply to a restricted amount of samples. A prominent symptom of overfitting is the generalization gap, defined as the difference between the validation and training error. A steady increase in this quantity is usually interpreted as the model entering the overfitting regime.

Figure 1a and 1b show a prototypical example of overfitting. We see in Figure 1a that **[accuracy on the training and validation set increase similarly during the early epochs of model training (1-10), with a low generalization gap. However, after this point, the accuracy on the validation set begins to decrease linearly as the training accuracy continues to increase toward a maximum around 96%, greatly increasing the generalization gap. This clearly demonstrates that the model starts to overfit as training progresses.]**

Figure 1b shows a similar pattern in cross-entropy error, though with a significantly steeper increase in the generalization gap over epochs. This is because, while accuracy measures the proportion of correct predictions, the cross-entropy error captures the precision of these predictions. This means that while not only identifying overfitting beginning around the 10th epoch, the error graph also shows how the model steadily and rapidly becomes more confident in its incorrect analysis of the data].

The extent to which our model overfits depends on many factors. For example, the quality and quantity of the training set and the complexity of the model. If we have sufficiently many diverse training samples, or if our model contains few hidden units, it will in general be less prone to overfitting. Any form of regularization will also limit the extent to which the model overfits.

2.1. Network width

[Question Table 1 - Fill in Table 1 with the results from your experiments varying the number of hidden units.] [Question Figure 2 - Replace the images in Figure 2 with figures depicting the accuracy and error, training

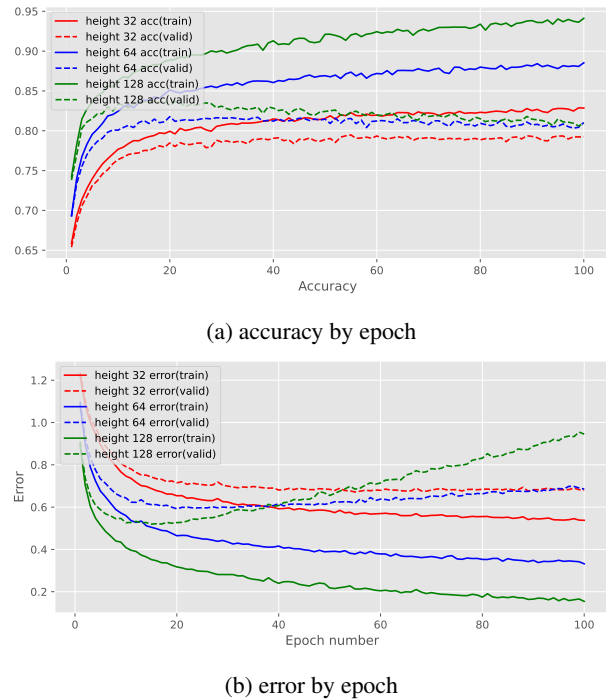


Figure 2. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network widths.

and validation curves for your experiments varying the number of hidden units.]

First we investigate the effect of increasing the number of hidden units in a single hidden layer network when training on the EMNIST dataset. The network is trained using the Adam optimizer with a learning rate of 9×10^{-4} and a batch size of 100, for a total of 100 epochs.

The input layer is of size 784, and output layer consists of 47 units. Three different models were trained, with a single hidden layer of 32, 64 and 128 ReLU hidden units respectively. Figure 2 depicts the error and accuracy curves over 100 epochs for the model with varying number of hidden units. Table 1 reports the final accuracy, training error, and validation error. We observe that **[overfitting is clearly displayed in Figure 2 by the models with 64 (blue) and 128 (green) hidden units, but not in the 32-wide model (red). Although showing a significantly higher cross-entropy error than the other models (see Table 1), the 32-wide model does not have a major increase in its generalization gap after 40 epochs. Training and validation values advance in parallel for both accuracy and error, with the final cross-entropy errors being much closer than in the 64/128-wide models. Furthermore, while approaching a lower training error faster than the other models, the 128-wide model has by far the largest generalization gap due to the steep gradient of its cross-entropy error once overfitting begins].**

[These observations are expected. The model with 32 hidden units performed poorly on the training data due to its limited flexibility: Without sufficient width

# Hidden Layers	Val. Acc.	Train Error	Val. Error
1	80.0	0.161	0.972
2	82.2	0.092	1.513
3	82.4	0.109	1.641

Table 2. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network depths on the EMNIST dataset.

to capture the complexity of the data, the model was unable to adequately classify samples from both the training and validation set. As described in Section 5.2 of (Goodfellow et al., 2016), this is a classic case of underfitting and demonstrates the importance of a model having enough capacity to fit the intricacies of the data.

Similarly, the overfitting of the wider models is also predictable. Without training being overseen by regularization techniques, the models are free to internalize niche features in the training data that are not descriptive of its general or true patterns. The model with 128 hidden units has the highest capacity to capture data properties and therefore shows the highest rate of generalization gap increase].

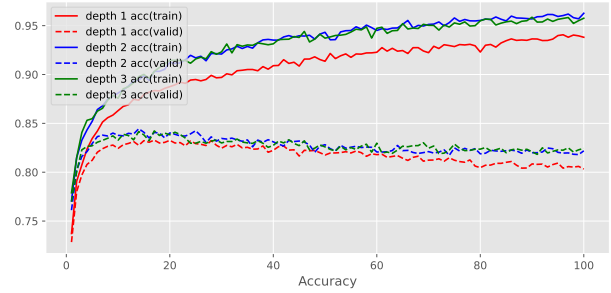
2.2. Network depth

[Question Table 2 - Fill in Table 2 with the results from your experiments varying the number of hidden layers.] [Question Figure 3 - Replace these images with figures depicting the accuracy and error, training and validation curves for your experiments varying the number of hidden layers.]

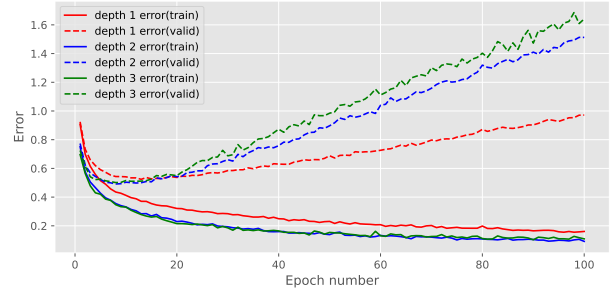
Next we investigate the effect of varying the number of hidden layers in the network. Table 2 and Figure 3 depict results from training three models with one, two and three hidden layers respectively, each with 128 ReLU hidden units. As with previous experiments, they are trained with the Adam optimizer with a learning rate of 9×10^{-4} and a batch size of 100.

We observe that [overfitting occurs at roughly the same epoch (10) as in the network width experiment, as seen in Figure 3a and 3b. Unlike in the width experiment, accuracy on the validation set decreases linearly from the same point ($\tilde{0.84}$) for all three network depths, with the values remaining within $\tilde{0.02}$ of each other for the rest of the training. However, this similar behaviour is not seen in the generalisation gap, with the cross-entropy error increasing rapidly for the 1 and 2 layer deep models in comparison to the 1 layer deep model. This results in the disparity in the validation error values seen in Table 2, with Val. Error of the single-layer model being significantly lower than the others.]

[As with the exploration of network width, increasing the number of parameters available to the model increases the severity of overfitting. With each additional



(a) accuracy by epoch



(b) error by epoch

Figure 3. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network depths.

hidden layer, the model has more capacity to capture complex functions that represent the data and a higher risk of capturing training-set-dependent functions. This results in the steeper gradients of validation error seen in the deeper models.

The more uniform behavior of training and validation accuracy, compared to changing model width, implies the dataset may not require the high model complexity provided by increasing depth in order to achieve high performance. The features themselves rather than the relations between them are likely more informative, so no major difference in model accuracy across depths is observed].

3. Regularization

In this section, we investigate three regularization methods to alleviate the overfitting problem, specifically dropout layers, the L1 and L2 weight penalties and label smoothing.

3.1. Dropout

Dropout (Srivastava et al., 2014) is a stochastic method that randomly inactivates neurons in a neural network according to an hyperparameter, the inclusion rate (*i.e.* the rate that an unit is included). Dropout is commonly represented by an additional layer inserted between the linear layer and activation function. Its forward pass during training is

defined as follows:

$$\text{mask} \sim \text{bernoulli}(p) \quad (1)$$

$$\mathbf{y}' = \text{mask} \odot \mathbf{y} \quad (2)$$

where $\mathbf{y}, \mathbf{y}' \in \mathbb{R}^d$ are the output of the linear layer before and after applying dropout, respectively. $\text{mask} \in \mathbb{R}^d$ is a mask vector randomly sampled from the Bernoulli distribution with inclusion probability p , and $\odot$ denotes the element-wise multiplication.

At inference time, stochasticity is not desired, so no neurons are dropped. To account for the change in expectations of the output values, we scale them down by the inclusion probability p :

$$\mathbf{y}' = \mathbf{y} * p \quad (3)$$

As there is no nonlinear calculation involved, the backward propagation is just the element-wise product of the gradients with respect to the layer outputs and mask created in the forward calculation. The backward propagation for dropout is therefore formulated as follows:

$$\frac{\partial \mathbf{y}'}{\partial \mathbf{y}} = \text{mask} \quad (4)$$

Dropout is an easy to implement and highly scalable method. It can be implemented as a layer-based calculation unit, and be placed on any layer of the neural network at will. Dropout can reduce the dependence of hidden units between layers so that the neurons of the next layer will not rely on only few features from of the previous layer. Instead, it forces the network to extract diverse features and evenly distribute information among all features. By randomly dropping some neurons in training, dropout makes use of a subset of the whole architecture, so it can also be viewed as bagging different sub networks and averaging their outputs.

3.2. Weight penalty

L1 and L2 regularization (Ng, 2004) are simple but effective methods to mitigate overfitting to training data. The application of L1 and L2 regularization strategies could be formulated as adding penalty terms with L1 and L2 norm square of weights in the cost function without changing other formulations. The idea behind this regularization method is to penalize the weights by adding a term to the cost function, and explicitly constrain the magnitude of the weights with either the L1 and L2 norms. The optimization problem takes a different form:

$$\text{L1: } \min_{\mathbf{w}} E_{\text{data}}(\mathbf{X}, \mathbf{y}, \mathbf{w}) + \lambda \|\mathbf{w}\|_1 \quad (5)$$

$$\text{L2: } \min_{\mathbf{w}} E_{\text{data}}(\mathbf{X}, \mathbf{y}, \mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad (6)$$

where E_{data} denotes the cross entropy error function, and $\{\mathbf{X}, \mathbf{y}\}$ denotes the input and target training pairs. λ controls the strength of regularization.

Weight penalty works by constraining the scale of parameters and preventing them to grow too much, avoiding overly

sensitive behaviour on unseen data. While L1 and L2 regularization are similar to each other in calculation, they have different effects. Gradient magnitude in L1 regularization does not depend on the weight value and tends to bring small weights to 0, which can be used as a form of feature selection, whereas L2 regularization tends to shrink the weights to a smaller scale uniformly.

3.3. Label smoothing

Label smoothing regularizes a model based on a softmax with K output values by replacing the hard target 0 labels and 1 labels with $\frac{\alpha}{K-1}$ and $1 - \alpha$ respectively. α is typically set to a small number such as 0.1.

$$\begin{cases} \frac{\alpha}{K-1}, & \text{if } t_k = 0 \\ 1 - \alpha, & \text{if } t_k = 1 \end{cases} \quad (7)$$

The standard cross-entropy error is typically used with these *soft* targets to train the neural network. Hence, implementing label smoothing requires only modifying the targets of training set. This strategy may prevent a neural network to obtain very large weights by discouraging very high output values.

4. Balanced EMNIST Experiments

[**Question Table 3 - Fill in Table 3 with the results from your experiments for the missing hyperparameter values for each of L1 regularisation, L2 regularisation, Dropout and label smoothing (use the values shown on the table).**]

[**Question Figure 4 - Replace these images with figures depicting the Validation Accuracy and Generalization Gap (difference between validation and training error) for each of the experiment results varying the Dropout inclusion rate, and L1/L2 weight penalty depicted in Table 3 (including any results you have filled in).**]

Here we evaluate the effectiveness of the given regularization methods for reducing the overfitting on the EMNIST dataset. We build a baseline architecture with three hidden layers, each with 128 neurons, which suffers from overfitting as shown in section 2.

Here we train the network with a lower learning rate of 10^{-4} , as the previous runs were overfitting after only a handful of epochs. Results for the new baseline (c.f. Table 3) confirm that lower learning rate helps, so all further experiments are run using it.

Here, we apply the L1 or L2 regularization with dropout to our baseline and search for good hyperparameters on the validation set. Then, we apply the label smoothing with $\alpha = 0.1$ to our baseline. We summarize all the experimental results in Table 3. For each method except the label smoothing, we plot the relationship between generalization gap and validation accuracy in Figure 4.

First we analyze three methods separately, train each over a set of hyperparameters and compare their best performing

Model	Hyperparameter value(s)	Validation accuracy	Train Error	Validation Error
Baseline	-	83.7	0.241	0.533
Dropout	0.6	80.7	0.549	0.593
	0.7	82.7	0.358	0.694
	0.85	85.1	0.329	0.434
	0.97	85.4	0.244	0.457
L1 penalty	5e-4	79.5	0.642	0.658
	1e-3	76.3	0.755	0.776
	5e-3	2.41	3.850	3.850
	5e-2	2.20	3.850	3.850
L2 penalty	5e-4	85.1	0.306	0.460
	1e-3	85.4	0.371	0.434
	5e-3	81.3	0.586	0.607
	5e-2	39.2	2.258	2.256
Label smoothing	0.1	83.3	0.847	1.235

Table 3. Results of all hyperparameter search experiments. *italics* indicate the best results per series (Dropout, L1 Regularisation, L2 Regularisation, Label smoothing) and **bold** indicates the best overall.

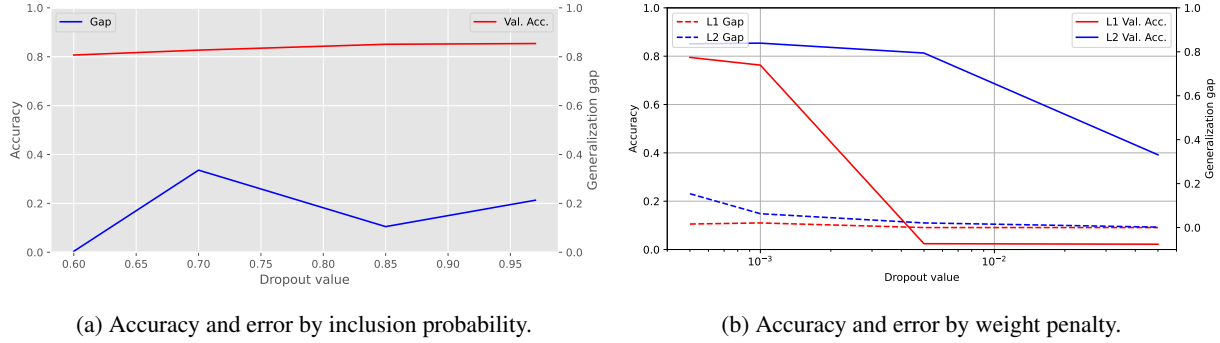


Figure 4. Accuracy and error by regularisation strength of each method (Dropout and L1/L2 Regularisation).

results.

[Dropout is trained on varying inclusion parameters between 0.6 and 0.97, with a single dropout layer placed after the second hidden layer. At lower inclusion probabilities (0.6 and 0.7), the series underperforms compared to the baseline in terms of accuracy due to the high information loss. By excluding a significant portion of nodes, the models overly regularize which results in underfitting during training. Additionally, these lower inclusion probabilities have vastly different generalization gaps that do not seem to produce an overall a linear pattern (as seen in Figure 4a). Though not addressed in (Srivastava et al., 2014), low inclusion probabilities can result in high instability during model training which leads to inconsistent results, as described in (Jain et al., 2015). This underperformance at low inclusion rates is mirrored at high rates, with the model performing its overall best at 0.97 (85.4% validation accuracy, 0.244 train error, 0.457 validation error).

To explore L1 behavior, regularization strength (applied to each trainable layer as a weight and bias penalty according to Formula 5) is ranged from 5e-4 to 5e-2,

with the best performance seen at 5e-4 (See Figure 4b). Though comparable to the baseline at this strength, it has worse values for both error and accuracy (79.5 for L1, 83.7 for baseline). All values of regularization strength produce low generalization gaps, with the higher values reducing the gap to 0 and showing its effectiveness at combating overfitting. However, high L1 values (5e-3, 5e-2) severely damages accuracy, underfitting compared to all other models. As a higher L1 strength favors lower weights, the model results in many weights being shrunk towards zero and leads it to be unable to capture anything significant within the data.

L2 regularization is explored with the same hyperparameters as L1, and demonstrates a similar but more moderate behavior to L1. Highest model performance is seen at lower regularization values but begins a steep decrease in accuracy at 5e-2 (Figure 4b), down to 39.2%. The generalization gap steadily improves towards zero as the regularization parameter increases, showing that this is a similarly effective approach to overfitting as L1 regularization.

Overall, Dropout and L2 regularization demonstrate the strongest performance for generalization, with equally high validation accuracies of 85.4% at hyperparameters of 0.97 and 1e-3 respectively. For this metric, L1 is less effective and leads to the model collapsing at high regularization values. Though showing unstable generalization gaps across inclusion probabilities, Dropout is at its worst competitive with the baseline model error. Both L1 and L2 regularization produce show excellent handling of overfitting, although in the case of L1 this is because the model performs equally poorly on training and validation data. Altogether, L2 with 1e-3 regularization performs the best in terms of both generalization and avoiding over/underfitting, with 85% Dropout also performing well but with a higher generalization gap compared to L2] .

[Given the opportunity to further explore model behavior, a potential objective could be to assess the performance of regularization techniques in the presence of noisy data:

Setup: Pre-process training data with addition of noise (Occlusion, Scaling, Pixel noise) (([Fan & Lee, 2021](#))). The exact nature of the noise may affect results in interesting ways but is not the focus of this experiment, as long as it is applied consistently across the training dataset. Model structure follows the baseline described above, and a baseline training instance can be run to evaluate the below trials:

1. Dropout (0.7) + L2 (5e-4): Does a moderate dropout and L2 maintain the balance of generalization overfitting, seen while separate, in the presence of noise?
2. Dropout (0.95) + L1 (1e-3): Can a moderate L1 penalty compensate for a higher dropout inclusion rate in reducing sensitivity to noise, or does this combination of techniques lead to underfitting?
3. Dropout (0.5) + L2 (1e-3): Does the best performing regularization improve robustness to noise when combined with low dropout, without introducing sparsity from the dropout?
4. Dropout (0.7) + L1 (5e-4) + L2 (5e-4): Compared to the previous training cases, does combining all three regularization techniques improve performance with noisy data while preserving model accuracy and generalization?

Another area to explore could be the effect of regularization on convergence speed. This could be done with little to no modification to the training experiments already described, but may provide useful insights into the efficiency of dropout, L1 and L2 regularization:

1. Dropout (0.95) + L2 (1e-4): Does reduced regularization improves convergence speed despite the risk

of overfitting, in particular for faster early-stage learning?

2. Dropout (0.7) + L2 (1e-3): A balanced approach of dropout and L2 regularization may control overfitting during training, but can it also achieve efficient convergence?
3. Dropout (0.7) + L1 (5e-3): Can a higher L1 regularization with moderate dropout reach a low generalization gap while maintaining convergence speed, or would it need training over increased epochs?
4. Dropout (0.5) + L1 (5e-4): Does a low inclusion probability with moderate L1 penalty control overfitting well while maintaining convergence speed, potentially by reducing dependence on individual neurons?

] .

5. Conclusion

[As seen in this report, overfitting should be a key consideration during model training. Though particularly pressing when handling models with high flexibility, its mitigation is not a straightforward process. Any attempt to contain model freedom introduces the risk of underfitting and reducing performance, as seen in the width and depth experiments. Dropout and Weight Penalty provide novel approaches to reduce overfitting while maintaining model accuracy, with L2 regularization being observed to produce the best balance for generalizing while avoiding underfitting. However, these also highlight the importance of managing regularization strength, demonstrated by the poor performance of L1 and L2 penalty at high strengths. Chapter 5 of ([Goodfellow et al., 2016](#)) provided the basis of this report, giving insight into both the problem of overfitting and potential approaches to the issue in later chapters, while ([Srivastava et al., 2014](#)) was used to formulate Dropout regularization. Further experimentation into the behavior of these techniques, specifically in the context of noisy data and training efficiency, is required to deepen the understanding and utility of regularization, as strategies for improving model performance remains an area of potential innovation] .

References

- Fan, Wan-Wei and Lee, Ching-Hung. Classification of imbalanced data using deep learning with adding noise. *Journal of Sensors*, 2021(1):1735386, 2021.
- Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- Jain, Prateek, Kulkarni, Vivek, Thakurta, Abhradeep, and Williams, Oliver. To drop or not to drop: Robustness, consistency and differential privacy properties of dropout. *arXiv preprint arXiv:1503.02031*, 2015.
- Ng, Andrew Y. Feature selection, l_1 vs. l_2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 78, 2004.
- Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1): 1929–1958, 2014.